

# Diffusion-MoE Phase 2 Training Investigation

## Findings report — all-layers training run, bugs found, and current state

Prepared for review · diffusion-moe project

---

### Executive Summary

Following Phase 1's diagnostics ( `phase1_findings_report.md` ) and the bounded single-layer pilot fine-tune, this phase asked the question those tools structurally couldn't answer: **what actually happens if every layer of the project's own from-scratch architecture is a `DiffusionMoELayer` , trained end-to-end, not just diagnosed or spliced into a frozen pretrained backbone?**

The honest answer is: six real, previously-undiscovered bugs, one badly-miscalibrated hyperparameter, and — after fixing all of them — clean, complete training runs that demonstrate the architecture can learn, route without permanently collapsing, and finish without crashing. That is real progress, not a finished result: a same-budget dense-model control, repeated across two seeds, decisively and consistently outperformed the all-layers diffusion-MoE configuration on quality, cost, *and* reliability (§6.4–6.5). See §7 for the full picture and what would change that conclusion.

1. **NaN divergence at step ~199** — `NystromDiffusionMap` 's kernel bandwidth ( `eps_` ) is fit once and frozen between landmark refits; once training moves the embedding distribution far enough, a token can drift outside every landmark's kernel support, and the unguarded division silently produces NaN, which `clip_grad_norm_` cannot repair. Fixed with both a defensive guard and — the deeper fix — a live `eps_` refresh between refits (§2.1).
2. **Checkpoint resume crashes a real MoE layer** — `NystromDiffusionMap` 's fitted state lives on a plain Python object, not an `nn.Module` , so it silently does not survive `state_dict()` save/load. Any resume off the exact refit-schedule boundary crashed outright. Found running the project's own `expert_attribution.py` tool against a real checkpoint, not by inspection (§2.2).
3. **No shared/always-active expert** — every FFN computation was contingent on sparse top-k routing; nothing had guaranteed, routing-independent gradient signal. Added a mandatory `shared_expert` , architecturally identical to a routed `ExpertFFN` , applied unconditionally outside the router (§3).
4. **Disk-full crash near the finish line** — unbounded checkpoint retention filled a 60GB instance's disk with ~5.7GB files, corrupting the final checkpoint write. Fixed with automatic pruning (§4).
5. **A catastrophic initialization bug** — `task_loss` started at ~823 nats against a random-guessing baseline of  $\ln(32000)=10.37$  , and never recovered below ~60 nats across any run, no matter how long training continued. Root-caused to two compounding, previously-unaddressed initialization gaps (no custom init existed anywhere in the model) and fixed; confirmed both in isolation and in a real training run (§5). This is the most consequential finding in this phase — everything observed in earlier runs (§6) was training on top of this bug.
6. **`mu_load` (the load-balance loss weight) was miscalibrated by ~2 orders of magnitude** relative to `task_loss` 's gradient once the loss scale was actually correct — measured directly, not guessed, and retuned (§6.2).
7. **`cfg.seed` was a no-op** — never applied to torch's global RNG anywhere in this pipeline (only `data.seed` , controlling shuffle order, was wired through), so model weight initialization — the dominant source of run-to-run variation — drew from whatever unseeded state the process happened to start in. Fixed, and what made the seed-repeat check in §6.5 meaningful in the first place (§9).

The final MoE run ( `graceful-universe-21` ) completed its full 2,441-step / 20M-token budget cleanly: zero NaN, `task_loss` ending at 7.08 (below the random baseline), all 24 layers' `load_loss` ending near zero (down from 7 of 24 near-total collapse mid-run), and `val_ppl` finally moving off its `exp(20)` safety cap for the first time in this investigation (7,286 → 5,081 → 35,294 spike → 16,158 → 3,245 → 2,239 → 974). That is real progress relative to every earlier run in this phase — but a same-budget **dense-model control run** ( `peachy-cloud-22` , §6.4), run immediately afterward, decisively beat it: `task_loss=4.80` , `val_ppl=93.2` , in ~15x less wall-clock time. A second seed of both configs (§6.5) confirmed this wasn't one lucky/unlucky comparison — dense stayed essentially unchanged (sub-1% variance), while the MoE run got worse on the second seed ( `val_ppl=1,478` , wall-clock 425 min) rather than better. See §7.

**§8 (added after the above was first written) goes further and identifies why.** A single, data-driven selective layer (§8.1) closes most of the all-layers gap on its own — but investigating *why* the routed layer still lagged dense surfaced a real bug ( `centroid_refresh_steps` was silently measured in micro-batches, refitting twice as often as configured, §8.2) and, more importantly, a structural finding: `task_loss` never recovers to its pre-refit level within a full refit-to-refit window (§8.3) — the diffusion map is fighting a representation that's continuously reshaped by the same gradient descent the routing depends on, not just suffering a brief coordinate-sign hiccup. A targeted fix for the latter (§8.4) helped only at the margin. The decisive test: `SwitchMoELayer` — already in this codebase, a standard continuously-trained linear router with no periodic refit at all — lands within a few percent of dense at the identical layer selection (§8.6), and two follow-up runs (§8.8) confirm this generalizes: Switch gets *closer* to dense when scaled to all layers (the opposite of diffusion, which got 4.6x worse), and is seed-robust like dense (sub-1% seed variance) rather than seed-sensitive like diffusion (44-52%). **This phase's honest bottom line, updated:** the infrastructure now works, and the diffusion-geometric router's underperformance is not primarily about sparse routing or this project's scale — it's the periodic-hard-refit design specifically, evidenced by a structurally different router closing the gap in its place across every axis tested. Per-project decision at the close of this phase: further diffusion-router development is on hold pending a real structural fix to the moving-target problem (§8.3), not just further mitigation of its symptoms (§8.4 already tried that).

---

## 1. Background and Motivation

Phase 1's diagnostics and the bounded pilot fine-tune ( `phase1_findings_report.md` §3.4-3.6) each tested one layer at a time, spliced into an otherwise-frozen pretrained Mistral-7B backbone. That structurally cannot answer a sharper question raised during review: *if a layer's router keeps forcing collapse onto one expert during real training, is that proof the layer can't benefit from diffusion routing at all, or just proof that one frozen-backbone, one-layer splice hasn't given the router anything to differentiate?* The only way to find out is to train the project's own from-scratch architecture with every layer routed, and watch. The user's explicit direction: *"I would let all layers use diffusion MoE and see what happens."*

This phase covers that run, the infrastructure it required (per-layer loss breakdown, checkpoint resume, expert-attribution analytics — see `methodology.md` §5 and §6 for the durable reference material), the bugs it surfaced, and the final result.

All runs in this section use the project's own 300M/24-layer from-scratch architecture ( `configs/model/300m.yaml` ), trained on `wikitext-103-raw-v1` via local parquet files (bypassing the recurring flaky-CDN Hub-streaming stall — see `methodology.md` §4), on a single rented RTX 3090 instance (vast.ai).

---

## 2. Bug: NaN Divergence, and the Deeper Fix Behind It

### 2.1 Root cause: a frozen kernel bandwidth under a moving distribution

The first all-layers run diverged to NaN at step 199, confirmed via an explicit `RuntimeWarning: invalid value encountered in divide` in `geometry/nystrom.py`. `NystromDiffusionMap.transform()`'s Nyström extension formula divides by each new point's total kernel mass to the (frozen) landmarks:

```
K_alpha = K_new / (d_alpha_new[:, None] * self._d_alpha_landmarks[None, :])
```

`eps_` (the kernel bandwidth) is derived once from the landmarks' own spread at the last refit (`fit()`), only run every `centroid_refresh_steps` and then frozen until the next one. If a token's embedding drifts far enough from that frozen reference — plausible once `centroid_separation_loss` has been actively pushing centroids apart for hundreds of steps — its kernel value to every landmark can underflow to exactly `0.0` in float64 (`exp(v)` underflows to the literal bit pattern `0.0` below roughly `v < -745`, not a small-but-nonzero value). The resulting `0/0` division produced `NaN`, which propagated through `task_loss / load_loss`, and — critically — `torch.nn.utils.clip_grad_norm` cannot repair a NaN gradient (the norm of a NaN tensor is itself NaN), so every parameter update after that point silently corrupted the entire model. Training continued to "run" for ~40 more steps on fully poisoned weights before crashing outright on an unrelated downstream `KMeans` call (sklearn's input validator rejects NaN explicitly; the raw numpy division earlier did not).

**Immediate fix:** guard the division (`d_alpha_new_safe`, mirroring the existing `row_sums` guard a few lines below) so a token with zero kernel mass gets diffusion coordinates at the origin (maximal uncertainty) instead of NaN.

## 2.2 The real fix: live-refreshing the kernel bandwidth between refits

The guard alone treats a symptom. The actual staleness problem — `eps_` describing a step-0 snapshot of the embedding distribution, growing more wrong the longer training runs between refits — remains. Fixed by having `transform()` re-derive `eps_` from the *current* batch's actual distance to the (still-frozen) landmarks on every call, reusing the distance matrix already computed for the Nyström kernel (cheap — no new k-means).

This surfaced two further correctness requirements, both needed for the fix to be mathematically sound rather than merely avoid a crash:

- **eigenvectors\_ / psi\_landmarks\_ must be re-derived alongside `eps_`** (extracted into a shared `_fit_landmark_spectrum`, called by both `fit()` and the refresh path): Nyström's defining correctness property — that applying the extension formula to the landmarks themselves must exactly reproduce their direct diffusion coordinates — only holds when the landmark spectrum and the new-point extension share one consistent kernel. Caught by `test_nystrom_quality_exact_reproduction_on_landmarks` when a first attempt (refreshing only `eps_` in isolation) broke it (0.0042 error against a  $1e-4$  tolerance).
- **diffusion\_eigenvectors 's ARPACK solve needed a seeded starting vector.** It previously ran once per landmark set, so its default unseeded random start never mattered. Once it could rerun every step, an unseeded start would make the diffusion-coordinate basis jitter step-to-step from solver randomness alone — independent of any real drift in the data, i.e. the opposite of what this fix was for.
- **fit\_transform 's own internal `transform()` call must skip the refresh** (new `_refresh_eps` parameter): it runs immediately after `fit()` on the same data, so there is no staleness yet to correct, and refreshing anyway would silently override `fit()`'s landmark-based heuristic and redo the eigensolve twice on every refit step for no benefit.

**Cost:** this is not free, but it is cheap relative to what it replaces. `eps_`'s refresh reuses an already-computed distance matrix; the re-derived eigensolve is over just the `n_landmarks × n_landmarks` matrix

(32–128 points), not another k-means clustering over the full token batch. Measured directly in production: steady-state per-step time rose from ~5.55s to ~7.0–7.3s (roughly 25–30%), on top of an already CPU-bound geometry pipeline (this entire subsystem runs on `z.detach()` 'd numpy/sklearn, never touching the GPU).

---

### 3. Feature: A Mandatory Shared Expert, Outside the Router

Once every layer was routed and training could be observed cleanly, a striking pattern emerged (visible even before the initialization bug was found, §5): `load_loss` stayed elevated across nearly all 24 layers simultaneously — not one collapsed outlier, a near-universal, if partial, imbalance (mean `load_loss` 3.21 across all layers at one snapshot, only 1 of 24 layers below 1.0). This raised the mechanistic question motivating this whole run: with every FFN computation contingent on sparse top-k routing, nothing in the network had guaranteed, routing-independent gradient signal — common/generic computation had to be either redundantly relearned by whichever expert won that step's routing lottery, or not learned reliably at all if routing was noisy.

This is a known pattern in production MoE architectures (DeepSeekMoE's "shared expert isolation," similarly Qwen2-MoE), added here as `DiffusionMoELayer.shared_expert` : architecturally identical to a routed `ExpertFFN` (same width, same `overlap_factor` ), applied unconditionally to every token, every step — no gate value, no centroid, entirely outside the diffusion router's influence. This adds real active compute per token ( `top_k + 1` experts fire instead of `top_k` ), not a reallocation of existing capacity.

Verified with a direct behavioral test

( `test_shared_expert_contributes_even_when_routing_contributes_nothing` ): with the routed dispatch path monkeypatched to contribute exactly zero, the layer's output still differs from the plain attention residual — the shared expert's contribution cannot be made routing-contingent by construction.

---

### 4. Bug: Disk-Full Crash Near the Finish Line

A run that had otherwise reached step 2,400 of 2,441 (98.3%) crashed on its own checkpoint save: the instance's 60GB disk filled completely (8 accumulated checkpoints × ~5.7GB each), and the final write was cut off mid-file ( `PytorchStreamWriter` failed writing file ; the resulting `step_2400.pt` was truncated to ~2.97GB against an expected ~5.68GB — corrupt, unusable). The last *valid* checkpoint ( `step_2100.pt` , saved before the disk filled) was confirmed to load cleanly.

Fixed with automatic checkpoint retention in `Trainer` : after each numbered checkpoint save, all but the most recent `keep_last_n_checkpoints` (default 2, configurable via `training.keep_last_n_checkpoints` ) are deleted. `best.pt` is a separate file and always kept regardless. `<= 0` disables pruning (old, unbounded behavior) for anyone who wants it.

---

## 5. The Initialization Bug: Task\_Loss Was Never Actually Calibrated

### 5.1 The anomaly

Every run before this fix showed the same signature: `task_loss` starting around 823 nats (step 1) and never recovering below roughly 60 nats, however long training continued. `val_ppl` ( `exp(min(avg_nll, 20.0))` ) was pinned at its safety cap — 485,165,195.4 exactly — at every single logged evaluation, across every run, all the way to the end of a 2,400-step run. This is a real red flag independent of routing behavior: this project's tokenizer (Mistral's, 32,000 tokens) gives a random-guessing baseline of `ln(32000) ≈ 10.37` nats — the cross-entropy a properly-initialized, completely untrained model should

sit near. A trained model that never gets within even a factor of 2 of that baseline, in either direction, is not "undertrained" — something is numerically broken.

### 5.2 Root cause, confirmed by direct measurement

No custom weight initialization existed anywhere in the model — every linear layer and the token embedding relied entirely on PyTorch's defaults. Two compounding gaps, confirmed empirically (not just inferred) by hooking every block of a freshly-constructed 24-layer model and measuring residual-stream norms and final logit magnitude directly:

- 1. `token_embedding` used `nn.Embedding` 's default init — `Normal(0, 1)` , `std=1.0` — far too large for a transformer embedding table, and especially costly because `tie_embeddings=True` by default makes this same oversized matrix double as the LM-head unembedding projection, directly inflating logit magnitude. This was the *dominant* cause: fixing only the second gap below (residual-output scaling) left final logits unchanged — actually slightly worse ( `max abs` 828 → 1071 ) — until this was also fixed.
- 2. **No depth-aware scaling on residual-branch output projections** (attention's `out_proj` , every FFN's `down_proj` — dense `TransformerBlock` , routed `ExpertFFN` , and the new `shared_expert` alike). Every serious transformer implementation (GPT-2, LLaMA, nanoGPT) scales these down by `1/sqrt(2 × n_layers)` at init specifically because residual-stream variance otherwise compounds with depth. Confirmed directly: residual-stream norm grew 32 → 44 over 24 layers unfixed; a controlled, smooth 1.0 → 7.1 fixed.

Combined fix: `token_embedding` initialized to the standard `std=0.02` (GPT-2/nanoGPT convention); every `out_proj` / `down_proj` weight scaled by `1/sqrt(2 × n_layers)` after construction (matched on the Linear's own attribute name, so it applies uniformly regardless of which module — dense, routed expert, or shared expert — it's nested in).

**Verified twice.** In isolation: a fresh random-init model's cross-entropy against random labels dropped from ~1,014 to **10.51** (essentially exactly `ln(32000)=10.37` ). In real training ( `vital-microwave-20` , the first run with this fix): step 1 `task_loss` = **10.61** , and a clean, sane trajectory afterward (10.61 → 9.03 → 8.47 by step 26) — nothing resembling any earlier run's trajectory.

### 5.3 What this means for everything observed before this fix

`resilient-brook-18` and `hardy-lake-19` (§6) both trained on top of this bug. Their qualitative conclusions (per-layer collapse patterns, shared-expert comparison) are likely still valid as *relative* signals within those runs, but their absolute loss numbers should not be treated as meaningful, and the shared-expert-vs-not comparison in §6.2 should be re-run now that the loss scale is correct before drawing any firm conclusion from it.

## 6. Run-by-Run Summary

| Run (wandb)        | Init fix | Shared expert | mu_load | Outcome                                                                                      |
|--------------------|----------|---------------|---------|----------------------------------------------------------------------------------------------|
| resilient-brook-18 | No       | No            | 0.01    | task_loss plateaued 60–110 nats; val_pp1 permanently capped; killed to add the shared expert |
| hardy-lake-19      | No       | Yes           | 0.01    | Modest improvement over the above at matched steps (task_loss=90.3 vs 100.3 @                |

|                                                                     |     |     |      |                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------|-----|-----|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                     |     |     |      | step 600) but still broken-scale; crashed on disk-full at step 2,400/2,441 (§4)                                                                                                                                                                                                           |
| vital-microwave-20                                                  | Yes | Yes | 0.01 | Step 1 <code>task_loss</code> =10.61 (correct!), but 18 of 24 layers collapsed to <code>load_loss</code> ≈7.0 (the exact CV <sup>2</sup> ceiling) by step ~600; <code>task_loss</code> climbed back to 11.5 — worse than its own starting point; killed for <code>mu_load</code> retuning |
| graceful-universe-21                                                | Yes | Yes | 2.0  | <b>Completed cleanly</b> — see §6.3                                                                                                                                                                                                                                                       |
| peachy-cloud-22 (dense control, <code>layers_to_replace=[]</code> ) | Yes | n/a | n/a  | Same budget, same init. <b>Beat graceful-universe-21 decisively</b> — see §6.4                                                                                                                                                                                                            |
| sunny-surf-23 (dense, <code>seed=123</code> )                       | Yes | n/a | n/a  | Near-identical to peachy-cloud-22 — see §6.5                                                                                                                                                                                                                                              |
| cool-pyramid-24 (MoE, <code>seed=123</code> )                       | Yes | Yes | 2.0  | Completed cleanly, but meaningfully worse and slower than graceful-universe-21 — see §6.5                                                                                                                                                                                                 |

## 6.1 The collapse dynamic got worse, not better, once the loss was fixed

`vital-microwave-20` is the sharpest result in this phase. Fixing the initialization did not just fix the loss scale — it let gradients flow cleanly for the first time. `task_loss`'s gradient became a strong, well-calibrated signal instead of one dominated by first fixing a catastrophic starting point, which made the router-collapse shortcut (route everything to one expert per layer, reducing the effective optimization problem) *more* attractive to find, not less. 18 of 24 layers reached essentially the exact `load_loss` ceiling (  $n\_experts - 1 = 7.0$  ) within ~600 steps. This is a direct escalation of the question this whole run was designed to answer: collapse is not a rare pathology in one unlucky layer, it is close to the *default* outcome once the network can actually learn efficiently, unless something resists it hard enough.

## 6.2 Measuring, not guessing, the fix

Rather than guess a new `mu_load`, the relative gradient pull of `task_loss` vs. `load_loss` on the shared `ExpertCentroids` parameter was measured directly on a fresh, correctly-initialized model (the parameter both losses actually compete to move, since `load_loss`'s gradient is confined to centroids — `Psi_t` is detached before the routing-loss computation, so it never reaches attention/backbone weights directly). Raw ratio: `task_loss`'s pull was only **1.86×** `load_loss`'s — a fair fight. The old `mu_load=0.01` shrank that to an effective **186×**, leaving `load_loss` almost no real ability to resist collapse — a value very likely tuned against the old, catastrophically-mis-scaled `task_loss` and never revisited once §5's fix corrected it. Set to `mu_load=2.0`, targeting an effective ratio near 1.

## 6.3 The final run

`graceful-universe-21` completed the full 2,441-step budget cleanly — zero NaN, `state=finished`. At the halfway point (step ~296), the picture was a real but partial improvement: 7 of 24 layers near collapse, versus 18 of 24 in the unfixed-`mu` run at a comparable point — down ~60%, not eliminated. By the *end* of training, the picture resolved further: all 24 layers' `load_loss` near zero (best 0.008, most below 0.0005) — though the path there was genuinely volatile, not a clean monotonic decrease ( `load_loss` oscillated between 0.45 and 3.6 across steps 1,200–2,200 before settling as the learning rate decayed toward its floor; see §7 for why this specific ending should not yet be over-interpreted).

`task_loss` ended at **7.08** — below the random baseline, genuine learning. `val_pp1` moved for the first time in this entire investigation, off the `exp(20)` cap:

```
step 300: 7,286
step 600: 5,081
step 900: 35,294  <- real spike, not a fluke
step 1200: 16,158
step 1800: 3,245
step 2100: 2,239
step 2400: 975   <- best of the run
```

Non-monotonic and still high in absolute terms for a 300M-parameter model (even on a small budget), but a real, measured signal that the model is generalizing at all — something no earlier run in this entire investigation had shown.

### 6.4 The dense-model control, and a decisive negative result

Immediately after `graceful-universe-21` completed, the same config was rerun with `layers_to_replace=[]` (dense baseline, identical data/budget/token count/hyperparameters otherwise) — the missing control flagged in every version of §7 until now. Same starting point confirmed ( `task_loss=10.59` at step 1 vs. the MoE run's `10.61` , so the initialization fix applies uniformly and this is a fair comparison):

|                                             | Dense ( <i>peachy-cloud-22</i> )                            | Diffusion-MoE, all layers ( <i>graceful-universe-21</i> )                                         |
|---------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Final <code>task_loss</code><br>(step 2441) | <b>4.80</b>                                                 | 7.08                                                                                              |
| Final <code>val_pp1</code>                  | <b>93.2</b>                                                 | 974.6                                                                                             |
| <code>val_pp1</code> trajectory             | Smooth, monotonic: 274 → 200<br>→ 156 → 131 → 100 → 95 → 93 | Noisy, huge spike at step 900: 7,286 →<br>5,081 → <b>35,294</b> → 16,158 → 3,245 → 2,239<br>→ 975 |
| Wall-clock, same<br>2,441 steps             | <b>20.1 min</b>                                             | 294.5 min (~4.9 hours, ~15x longer)                                                               |

The dense model wins decisively on every axis, at matched token budget. The time gap is almost entirely the CPU-bound Nyström/eigensolve pipeline (~7 s/step MoE vs. ~0.49 s/step dense — matches the ~15x wall-clock ratio directly), not GPU compute. This is not even a "worse quality, but cheaper active compute" tradeoff in MoE's favor: the diffusion-MoE model's active FFN width per token ( `top_k+1=3` experts × 512 each = 1,536) is *smaller* than the dense model's full 4,096-wide FFN — less active capacity, ~15x the wall-clock cost, and a meaningfully worse result.

**This is a real, decisive answer, and it is unfavorable to the current approach.** Nothing in this investigation shows the added complexity of diffusion-routed MoE buying anything at this scale/budget. The standard framing for why sparse MoE is worth its complexity (Clark et al. 2022's scaling-law work, referenced earlier in this investigation) is specifically about quality-per-active-FLOP *at scale* — 20M tokens on a 300M-parameter model is nowhere near the regime that literature has actually demonstrated an advantage in. This result is consistent with that: it does not show diffusion-MoE is a bad idea in general, but it does mean this specific comparison offers no evidence *for* it, and meaningful evidence against training it further at this scale without a specific, articulated reason to expect the gap to close only at larger scale.

## 6.5 Seed-repeat check: dense is consistent, diffusion-MoE is not

Both §6.4 configs were rerun with `seed=123` (a new fix this phase: `cfg.seed` was previously never actually applied to torch's global RNG — only `data.seed`, controlling shuffle order, was wired through, meaning every prior run's model initialization was drawn from whatever unseeded state the process happened to start in, not a controlled value at all — see §9).

|                       | seed=42   | seed=123  | Spread  |
|-----------------------|-----------|-----------|---------|
| Dense task_loss       | 4.80      | 4.80      | ~0%     |
| Dense val_ppl         | 93.2      | 92.8      | <1%     |
| Dense wall-clock      | 20.1 min  | 20.0 min  | ~0%     |
| MoE task_loss         | 7.08      | 7.44      | ~5%     |
| MoE val_ppl           | 974.6     | 1,477.5   | ~52%    |
| MoE wall-clock        | 294.5 min | 424.6 min | ~44%    |
| MoE load_loss (final) | 0.00044   | 0.00020   | both ≈0 |

Two things confirmed here. First, §6.4's dense-beats-MoE result is not a fluke of one lucky/unlucky seed pair — dense is close to seed-invariant, and the *worse* of the two MoE seeds (`seed=123`, `val_ppl=1,478`) is still over 15x worse than *either* dense run. Second, and a genuinely new finding rather than just confirmation: diffusion-MoE is not just worse on average, it is meaningfully *less consistent* than dense, on both quality and cost — a 52% perplexity swing and a 44% wall-clock-time swing between two runs that differ only in random seed, versus dense's sub-1% variation on both. This extends Phase 1 §3.5's seed-42-vs-123 finding (different experts collapsing depending on seed) rather than just replicating it: it's not only *which* expert collapses that's seed-sensitive, it's the *total training cost and final quality* too — a real, separate cost of the current approach's variance, independent of its average performance gap.

## 6.6 What token-level routing actually looks like (early-training snapshot)

`scripts/expert_attribution.py` (extended this phase with Hydra CLI override support, so it can point at whatever data config a checkpoint was actually trained against — see `methodology.md` §6) was run against `hardy-lake-19`'s step-300 checkpoint. Filtering out padding (a first pass mistakenly included it — 479 of 492 positions in the sampled sequence were padding, trivially uniform) revealed real, reproducible structure, but **mostly positional, not semantic**, at this early stage: layer 0 routes only the first 1–4 tokens of every sequence to one expert (sequence-start detection) and everything else to another, regardless of topic; layer 6 shows a clean depth-progression through three experts as a sequence gets longer, independent of content; layer 20 was fully collapsed (one expert, ~99% of all real tokens across 5 sampled sequences). One layer (7) showed a plausible content-correlated split — a block of numeric/statistical text routing differently than surrounding narrative prose — but with one sequence and no position control, this could not be distinguished from coincidence.

## 6.7 Which layers to select for a follow-up: a data-driven answer, this architecture's own

§7's revised recommendation points toward a *selective* layer-replacement strategy rather than all-layers. Phase 1's oracle-ceiling layer indices (10/29/30 recoverable; 17/20/31 needing cosine) don't transfer — that was a 32-layer Mistral-7B model, this is a 24-layer from-scratch one. This section derives candidates from this architecture's own data instead, combining both completed MoE runs (§6.5).

Per-layer `load_loss` , averaged across each run's full trajectory (not just the final point), both seeds:

| Layer           | seed=42 mean | seed=123 mean | Average      | Seed-to-seed spread            |
|-----------------|--------------|---------------|--------------|--------------------------------|
| 0               | 0.223        | 0.206         | <b>0.214</b> | <b>0.017</b>                   |
| 1               | 0.901        | 1.328         | 1.114        | 0.427                          |
| 2               | 0.769        | 1.701         | 1.235        | 0.932                          |
| 4               | 1.792        | 1.890         | 1.841        | <b>0.098</b>                   |
| 17              | 2.344        | 2.159         | 2.252        | <b>0.185</b>                   |
| 21              | 3.909        | 3.743         | 3.826        | <b>0.167</b>                   |
| 20              | 3.328        | 3.601         | 3.464        | 0.273                          |
| 22              | 4.182        | 3.194         | 3.688        | 0.988                          |
| 23              | 4.264        | 3.307         | 3.786        | 0.957                          |
| 3, 5, 9, 14, 19 | —            | —             | mid-range    | <b>1.0–1.6 (most volatile)</b> |

(Remaining 13 layers omitted for space; the full 24-row table is reproducible from the wandb API, see §6.5's query.) Two independent signals matter here, not one: a low *average* means a layer tends to stay balanced; a low *spread* means that's reliable rather than a coin flip. Layer 0 dominates on both. Layers 4, 17, and 21 are a clear second tier — moderate-to-high averages, but consistently so (low spread) in both seeds. Layers 3/5/9/14/19 have seed-to-seed spreads of 1.0–1.6 — even where their average looks tolerable, they swing unpredictably and are poor candidates regardless of mean.

**Confirmed against the final checkpoints directly**, not just the training-average proxy —  
`expert_attribution.py` 's top-expert-share on both runs' `step_2400.pt` :

| Layer | seed=42 top share        | seed=123 top share       |
|-------|--------------------------|--------------------------|
| 0     | 63.9%                    | 58.0%                    |
| 4     | 78.6%                    | <b>91.9% (collapsed)</b> |
| 17    | <b>91.4% (collapsed)</b> | <b>97.3% (collapsed)</b> |
| 20    | 64.2%                    | <b>97.9% (collapsed)</b> |
| 21    | 69.2%                    | <b>97.3% (collapsed)</b> |

Layer 0 is the only layer that stayed meaningfully balanced on *both* seeds at the final checkpoint — everything else that looked reasonable on seed=42 (4, 20, 21) collapsed hard on seed=123. This matches the training-average table's ranking exactly and is the strongest, most consistent signal in this investigation for which single layer to pick first.

**What layer 0 actually learned, at the token level (padding filtered):** on `graceful-universe-21` 's (seed=42) final checkpoint, layer 0 shows a clean, token-by-token split correlating with a real linguistic feature — numeric/digit tokens route to one expert, word tokens to another, consistently across all 5 sampled sequences (e.g. `2:5 3:5 @:5 .:5` vs. `the:1 city:1 population:1 was:1` in the same sequence). Layer 4 shows the same categorical split with the expert IDs swapped. The collapsed layers

show either nothing (layer 17: ~everything to one expert, no structure at all) or a noisier, degraded version of the same digit/word signal (layers 20/21: mostly consistent, with real exceptions).

**This does not fully reproduce on cool-pyramid-24 's (seed=123) final checkpoint**, and the way it fails to reproduce is itself informative: layer 0 stays balanced (confirming the robustness finding above), but the *specific* boundary it draws is different — mostly one expert throughout, with a second expert taking over specifically in the numeric-dense passage plus scattered periods/conjunctions, rather than seed=42's clean per-token digit/word alternation. **The honest reading:** layer 0 reliably learns to *specialize rather than collapse* — that part is robust across seeds, confirmed three independent ways here (training-average `load_loss`, its spread, and final-checkpoint top-share). *What* it specializes on is not a fixed, guaranteed feature — the router reliably finds *some* real distinction to exploit, not always the *same* one.

**Recommendation for the next run:** `layers_to_replace=[0]` as the single best-evidenced candidate, with `[0, 4, 17, 21]` as a secondary option if testing more than one layer is worth the added complexity — those three are the next tier by both average and consistency, despite 4 and 21 each collapsing on one of the two seeds tested. The five volatile mid-range layers (3, 5, 9, 14, 19) should be avoided in either case; their unpredictability makes them poor building blocks for a configuration meant to be reliable.

---

## 7. Current State and What Would Be Needed Before Scaling Up

**What this phase established:** the infrastructure-level failure modes are found and fixed — NaN divergence, checkpoint resume, disk exhaustion, and (the big one) the initialization bug that made every prior run's absolute numbers meaningless. One full run now completes cleanly end-to-end with sane losses throughout. Now that the missing control has been run (§6.4) *and* repeated with a second seed (§6.5): **at this scale and budget, the dense baseline beats the all-layers diffusion-MoE model decisively and consistently — better `task_loss`, an order-of-magnitude-plus better `val_ppl` on both seeds, and 15-21x less wall-clock time — while also being far more seed-consistent than the MoE configuration (sub-1% run-to-run variance vs. MoE's 44-52%)**.

**Remaining open questions:**

1. **The final near-zero `load_loss` coincides with the learning rate being fully decayed, on both seeds.** It is plausible routing balance was actively, robustly resolving itself in both cases — and equally plausible it simply froze wherever it happened to be as gradients vanished toward the end of the cosine schedule, which would not be evidence of a stable equilibrium at all. Two data points now both showing the same end-of-run pattern makes the "real equilibrium" reading somewhat more credible than after §6.3 alone, but doesn't settle it.
2. **20M tokens is a smoke-test budget**, not a real pretraining scale, and the standard case for why sparse MoE is worth its complexity (Clark et al. 2022's routed-vs-dense scaling laws, discussed earlier in this investigation) is specifically about quality-per-active-FLOP *at scale* — this comparison is nowhere near the regime that literature has actually demonstrated an advantage in. §6.4/§6.5's results do not show diffusion-MoE is a bad idea in general; they show this specific comparison offers no evidence for it, twice.

**Recommendation, revised again:** not ready for a scaled-up run, and the seed-repeat result (§6.5) removes the most likely objection to that conclusion — this was not one unlucky comparison. §8 (added after this section was first written) substantially sharpens the picture further: a selective single layer (§8.1) closes most of the all-layers gap, but the deeper cause is a moving-target problem in the diffusion router's periodic hard refit (§8.3), not sparse routing itself or this project's scale — confirmed by `SwitchMoELayer`, a structurally different (continuously-trained, no-refit) router, landing within a few percent of dense at the identical layer selection (§8.6). The practical recommendation this points to: before investing further in the diffusion-geometric router specifically, its cost (still ~70% slower than Switch even at one layer, from the

CPU-bound Nyström pipeline) needs either a decisive quality advantage over Switch to justify it, or a fix to the moving-target problem itself (not just its symptoms, per §8.4) — neither of which this phase found. Switch is the stronger candidate for anything built on top of this codebase going forward unless one of those changes.

## 8. Selective-Layer Results, the Refit-Discontinuity Investigation, and a Structural Alternative

Following §7's revised recommendation, this section covers what actually running §6.7's data-driven layer candidates showed, a real bug found while investigating why the routed layer wasn't recovering as expected, a deeper structural finding about *why* the recovery is incomplete even with that bug fixed, and the resulting decision to test a structurally different router as the next comparison.

### 8.1 Selective-layer results

Both of §6.7's candidates were run at the same 20M-token budget, `seed=42`, with the fixed initialization:

| Config                                   | task_loss   | val_pp1      | Wall-clock |
|------------------------------------------|-------------|--------------|------------|
| Dense                                    | 4.80        | 93.2         | 20.1 min   |
| <b>Selective [0]</b> (polar-leaf-25)     | <b>5.56</b> | <b>210.1</b> | 34.8 min   |
| Selective [0,4,17,21] (smooth-donkey-26) | 8.06        | 2,885.6      | 83.3 min   |
| All-layers MoE                           | 7.08        | 974.6        | 294.5 min  |

One well-chosen layer closes most of the gap to dense on every axis ( `val_pp1` ~4.6x better than all-layers, wall-clock ~8.5x faster) — real support for the selective-layer direction over all-layers replacement. Adding the three second-tier candidates made things *worse*, not better — worse than [0] alone and worse than all-layers on `val_pp1`. This is a correction to §6.7's own framing: layers 4/17/21 were recommended based on *low seed-to-seed variance* in their `load_loss`, but their *average load\_loss* was still moderate-to-poor (1.8–3.8, nowhere near layer 0's 0.21) — reliably mediocre is not the same as good, and diluting the one genuinely well-balanced layer with three consistently-so-so ones hurt more than it helped. Layer 0 alone remains the best MoE configuration found in this investigation.

### 8.2 Bug: `centroid_refresh_steps` was measured in micro-batches, not steps

Found by a sharp observation during live monitoring: a repeating loss-disruption pattern visible at 250-step intervals that didn't match the configured `centroid_refresh_steps=500` at all. Root cause:

`DiffusionMoELayer`'s internal `_step` counter increments once per `forward()` call, but `Trainer.train_step()` calls `forward()` once per *micro-batch* — `grad_accum_steps` times per logged/optimizer step, not once. Every other `*_steps` config value ( `checkpoint_steps`, `eval_steps`, `log_steps` ) is measured in optimizer-step units; `centroid_refresh_steps` silently wasn't. With this project's `grad_accum_steps=2` default, **every run in this investigation had been refitting landmarks twice as often as configured — 8 times over a 2,441-step run, not 4.**

Confirmed directly: the same crash-then-recover `load_loss` / `task_loss` signature already found at steps 500/1000/1500/2000 (§2.1) also appears identically at 250/750/1250/1750 in `graceful-universe-21` — exactly the halved-cadence prediction.

Fixed by threading `grad_accum_steps` into `DiffusionMoELayer` (and the full `_build_moe_layer / DiffusionMoETransformer / build_model_from_config` chain), so the refresh check is `_step % (centroid_refresh_steps * grad_accum_steps) == 0` — robust to future changes to `grad_accum_steps`, rather than requiring it to be manually doubled by hand.

### 8.3 The deeper finding: the model doesn't recover before the next refit hits

Investigating the disruption further (prompted by the question of whether a single refit's damage is transient or lasting) surfaced something worse than §2's original characterization. Tracing `task_loss` across a *full* refit-to-refit window in `graceful-universe-21` (step 250 to step 750):

```
step 230 (pre-refit): task_loss=6.92
step 270 (post-refit): task_loss=9.62  <- spike
step 490 (mid-window): task_loss=7.84  <- best it gets, still worse than pre-refit
step 750 (next refit): task_loss=8.45  <- never recovered, then hit again
```

`task_loss` never returns to its pre-refit level anywhere in that 500-step window, and the *next* refit lands before it can close the remaining gap. §2.1's original claim ("`load_loss` recovers within ~10-15 steps") was real but incomplete: that described `load_loss`'s aggregate *magnitude* returning to a normal noisy range, not the model's actual specialization quality recovering — the two were conflated. The honest framing: the diffusion map assumes something close to a stable manifold to fit a geometric embedding of, but the thing being embedded (post-attention activations) is itself continuously reshaped by the same gradient descent process the routing depends on (`task_loss`'s gradient flows into the attention weights that produce those activations; only the routing losses are detached — see `methodology.md` §5.2). A periodic *hard* refit repeatedly asks the router to re-orient to wherever that target has drifted to, with experts caught mid-specialization each time. This has a direct parallel in the deep-clustering literature, where alternating between fitting a clustering and updating the representation it's fit to is a known source of instability.

### 8.4 The sign-alignment fix helped only at the margin

A candidate mitigation: diffusion-map eigenvectors are only defined up to sign, and a refit replaces the basis outright (fresh landmarks, fresh eigensolve) with no guaranteed relationship to the old basis's orientation — `ExpertCentroids` doesn't get remapped when this happens, so an arbitrary sign flip adds a second, gratuitous discontinuity on top of the real drift from §8.3. Fixed in `NystromDiffusionMap.fit()`: snapshot the old basis's embedding of the current batch before overwriting anything, then flip each new component's sign to agree with it (confirmed against real data: a natural refit reproducibly flips a component's sign without the fix, dot product `-0.0043` against the pre-refit embedding, and does not with it).

Tested head-to-head on the best selective config (`layers_to_replace=[0]`, `seed=42`): `polar-leaf-25` (before) vs. `denim-darkness-27` (after).

|                                                               | Before   | After    |
|---------------------------------------------------------------|----------|----------|
| <code>task_loss</code>                                        | 5.56     | 5.57     |
| <code>val_ppl</code>                                          | 210.1    | 211.0    |
| Wall-clock                                                    | 34.8 min | 34.8 min |
| Peak <code>task_loss</code> in the step-250 disruption window | 7.60     | 7.15     |

Essentially no change to the final outcome, though a small, real softening of the immediate disruption (peak task\_loss 7.60 → 7.15). This is exactly what §8.3's framing predicts: sign-alignment removes a secondary source of discontinuity, but the dominant cause — the representation itself moving under the router — is untouched by it. Kept (it's a correct, cheap, structural fix, and it does measurably soften the immediate jolt even if it doesn't close the gap), but not a solution to the underlying problem on its own.

### 8.5 The shared expert, made a uniform toggle

Originally added only to `DiffusionMoELayer`, unconditionally on (§3). Made a `use_shared_expert` config toggle ( `routing.use_shared_expert`, default `true` ) threaded uniformly across all four router variants ( `diffusion` / `switch` / `cosine` / `random` ), so it can be compared on/off consistently rather than assumed to help everywhere, and so the comparison in §8.6 holds it constant rather than present on only one side.

### 8.6 A structurally different router: Switch Transformer's learned linear gate

§8.3's finding reframes the problem: the issue isn't primarily the diffusion router's refit *mechanics* (§8.2/ §8.4 both fixed real bugs there with only marginal effect) — it's that periodic hard refitting of a geometric embedding is fighting a representation that's continuously moving under gradient descent.

`SwitchMoELayer` — already implemented in this codebase as a baseline ( `router: "switch"` ), a single learned linear projection to expert logits, trained continuously by ordinary backprop, no diffusion map, no periodic refit, no discontinuity by construction — sidesteps this entire class of problem rather than mitigating it. Tested at the same config as the best diffusion result ( `layers_to_replace=[0]`, `use_shared_expert=true`, `seed=42` ):

| Config                                | task_loss   | val_ppl      | Wall-clock      |
|---------------------------------------|-------------|--------------|-----------------|
| Dense                                 | 4.80        | 93.2         | 20.1 min        |
| <b>Switch [0] (efficient-hill-28)</b> | <b>4.93</b> | <b>105.3</b> | <b>20.2 min</b> |
| Diffusion [0] (polar-leaf-25)         | 5.56        | 210.1        | 34.8 min        |
| Diffusion all-layers                  | 7.08        | 974.6        | 294.5 min       |

**Decisive, and it answers the question this section opened with.** Switch is nearly indistinguishable from dense — `task_loss` within 0.13 nats, `val_ppl` within 12, wall-clock within 6 seconds — while diffusion routing at the identical layer selection is ~2x worse on `val_ppl` and 70% slower even at its best configuration found in this investigation. This is a clean, well-controlled comparison: identical layer, identical shared-expert setting, identical seed, only the router mechanism differs. The diffusion router's problems are not primarily "sparse routing is hard at this scale" — a structurally different router (continuous, gradient-trained, no periodic refit) closes almost the entire gap to dense on its own. The periodic-hard-refit design (§8.3's moving-target problem) is the dominant cause of the diffusion router's underperformance relative to both dense and to a standard sparse-MoE alternative, not sparse routing in general and not this project's small scale.

### 8.7 Observability gap: wandb never logged the actual run config

Found while verifying the Switch comparison (§8.6) was actually apples-to-apples with the diffusion runs on `top_k / n_experts`: `Trainer.__init__` called `wandb.init(project=...)` with no `config=` argument at all, for every run in this entire investigation. None of a run's actual hyperparameters — `routing.top_k`, `model.router`, `use_shared_expert`, any of it — were ever recorded in wandb. Confirming what a given run actually used required SSHing into the instance and reading its log file by hand

(and even that was often unavailable in the moment: `nohup` 's stdout redirection is block-buffered, so the printed config frequently hadn't flushed to disk yet for a still-running process — the same buffering behavior noted throughout this investigation's live-monitoring sections).

Fixed: `wandb.init(..., config=wandb_config)`, where `wandb_config` is `OmegaConf.to_container(config, resolve=True)` for a real Hydra `DictConfig` (flattened into a plain, JSON-serializable dict — passing the `DictConfig` object through as-is would not have logged correctly), falling back to the config object as-is for the plain-dict configs the test suite uses. Every run from this point forward has its actual config in wandb's own config panel — no more reconstructing it from launch commands or SSH sessions after the fact.

## 8.8 Switch generalizes: all-layers scaling and seed-robustness, both confirmed

Two follow-up runs, both `use_shared_expert=true`, `top_k=2`:

| Config                                                | task_loss   | val_ppl      |
|-------------------------------------------------------|-------------|--------------|
| Dense                                                 | 4.80        | 93.2         |
| <b>Switch, all layers</b> (faithful-jazz-29, seed=42) | <b>4.87</b> | <b>100.4</b> |
| Switch [0] (efficient-hill-28, seed=42)               | 4.93        | 105.3        |
| Switch [0] (winter-moon-31, seed=123)                 | 4.90        | 104.9        |
| Diffusion [0] (polar-leaf-25)                         | 5.56        | 210.1        |
| Diffusion all-layers (graceful-universe-21)           | 7.08        | 974.6        |

Two results, both favorable to Switch and both a sharp contrast with diffusion's behavior at the same two axes:

- **Scaling to all layers helps Switch, not hurts it.** All-layers Switch ( `val_ppl` 100.4) is closer to dense (93.2) than layer-0-only Switch is (105.3) — the opposite of diffusion, where going from one layer (210.1) to all layers (974.6) made things roughly 4.6x worse. Switch's continuously-trained linear gate doesn't accumulate the periodic-refit disruption (§8.3) that compounds across layers for the diffusion router; more routed layers is just more useful sparse capacity.
- **Seed-robustness matches dense, not diffusion.** `efficient-hill-28` (seed=42) and `winter-moon-31` (seed=123) land within 0.6% of each other on `task_loss` and 0.4% on `val_ppl` — comparable to dense's sub-1% seed variance (§6.5), and nowhere near diffusion's 44-52% seed-to-seed swings at the same layer selection.

Together with §8.6, this closes out the comparison cleanly across every axis tested — single-layer quality, all-layers scaling, and seed stability — and all three favor Switch over the diffusion router by a wide, consistent margin. Combined with the user's separate observation that neither the standard CV<sup>2</sup> load-balance loss ( `coefficient_of_variation_loss` ) nor the Switch auxiliary loss ( `switch_load_balance_loss` ) penalizes anything beyond marginal per-expert dispatch frequency — a router that consistently pairs experts into redundant duplicate groups (e.g. always routing to {0,1} or {2,3} out of 4) would score as perfectly "balanced" under either loss, since both are satisfied by symmetric duplication just as well as by genuine diversity — this is flagged as an open question for any further Switch-based work, not yet checked empirically (no orthogonality/diversity term exists anywhere in this codebase, across any of the four router variants).

(Note: the two follow-up runs launched back-to-back initially collided on the single-GPU instance — the second crashed with a CUDA OOM while the first was still resident — and had to be resequenced. Unrelated to the architecture; noted here only because it's a recurring operational hazard on a single-GPU box.)

## 9. Code Changes Summary

| File                                                                                     | Change                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| geometry/nystrom.py                                                                      | Guarded the <code>d_alpha_new</code> division against exact-zero underflow; added live <code>eps_/landmark-spectrum</code> refresh in <code>transform()</code> (new <code>_refresh_eps</code> param, default on); extracted <code>_fit_landmark_spectrum</code> shared between <code>fit()</code> and the refresh path; <code>fit()</code> now sign-aligns the freshly-fit eigenbasis to agree with the previous one's embedding of the same batch (§8.4) |
| geometry/eigensolver.py                                                                  | <code>diffusion_eigenvectors</code> gained a <code>random_state</code> param, seeding ARPACK's starting vector — needed once the eigensolve could rerun mid-training, not just once per landmark set                                                                                                                                                                                                                                                      |
| models/moe_layer.py                                                                      | Added <code>shared_expert</code> (outside the router; §3); <code>_compute_diffusion_coords</code> now also refits when <code>ndm.landmarks_</code> is <code>None</code> (checkpoint-resume fix), not only on the step-count schedule; refresh cadence now accounts for <code>grad_accum_steps</code> (§8.2); <code>use_shared_expert/grad_accum_steps</code> constructor params                                                                           |
| models/switch_moe_layer.py,<br>models/cosine_moe_layer.py,<br>models/random_moe_layer.py | Added the same <code>use_shared_expert</code> toggle (§8.5) — the shared expert is no longer <code>DiffusionMoELayer</code> -only                                                                                                                                                                                                                                                                                                                         |
| models/moe_model.py                                                                      | Fixed <code>token_embedding</code> init ( <code>std=0.02</code> , was <code>nn.Embedding</code> default <code>std=1.0</code> ); added depth-aware <code>1/sqrt(2×n_layers)</code> scaling to every <code>out_proj/down_proj</code> weight after construction; threads <code>grad_accum_steps/use_shared_expert</code> through <code>_build_moe_layer</code>                                                                                               |
| models/model_factory.py                                                                  | Reads <code>training.grad_accum_steps</code> and <code>routing.use_shared_expert</code> from config                                                                                                                                                                                                                                                                                                                                                       |
| training/trainer.py                                                                      | Added non-finite-loss guard in <code>train_step</code> (raises <code>FloatingPointError</code> before <code>backward()/optimizer.step()</code> instead of silently training on <code>NaN/Inf</code> ); added <code>_prune_old_checkpoints</code> ( <code>keep_last_n_checkpoints</code> , default 2); <code>wandb.init</code> now actually passes <code>config=</code> (§8.7) — was never logging any run's real hyperparameters at all                   |
| data/dataloader.py                                                                       | Fixed <code>local_data_files</code> reaching the real Hydra-config-driven training path — a <code>ConfigAttributeError</code> several frames deep ( <code>omegaconf.ListConfig</code> vs. plain <code>list</code> ), not an obvious error at the call site                                                                                                                                                                                                |
| scripts/expert_attribution.py                                                            | Added Hydra-style CLI override support ( <code>parse_known_args</code> ), so it can point at the real data config a checkpoint was trained against instead of always falling back to <code>base_config.yaml</code> 's defaults                                                                                                                                                                                                                            |
| scripts/train.py                                                                         | <code>cfg.seed</code> now actually applied<br>( <code>torch.manual_seed/np.random.seed/random.seed/torch.cuda.manual_seed_all</code> )                                                                                                                                                                                                                                                                                                                    |

|                          |                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                          | before model construction — previously a no-op                                                                                             |
| configs/base_config.yaml | routing.mu_load: 0.01 → 2.0 (measured, §6.2);<br>training.keep_last_n_checkpoints: 2 (new); routing.use_shared_expert:<br>true (new, §8.5) |

All changes are covered by tests reproducing the specific failure before confirming the fix, not just testing the fix in isolation — e.g. a mixed-batch outlier reproducing the exact NaN-underflow condition; a cross-object checkpoint resume at a step deliberately off the refit schedule, confirmed to crash on the pre-fix code (verified by temporarily reverting the fix) and pass after; a monkeypatched zero-contribution routed path confirming the shared expert's contribution is structurally unconditional; a real disk-bounded checkpoint-pruning scenario; the random-label cross-entropy check confirming init-time loss lands near the true random baseline (confirmed to fail — 61.5 vs. an expected <13.8 — on the pre-fix code); a same-seed-reproduces/different-seed-diverges check on model initialization; a `centroid_refresh_steps / grad_accum_steps` interaction check confirming the correct (not halved) refit cadence; and a sign-alignment check confirming a real refit reproducibly flips a component's sign without the fix and does not with it. Full suite: 385 tests passing at last check.

---

## Appendix: Open Questions Carried Forward

- Does the depth-progression / sequence-start routing pattern (§6.6) persist, sharpen into content-sensitivity, or dissolve entirely at a later training step and a larger token budget? **Partially answered for layer 0 by §6.7:** by the final checkpoint it had moved past pure position into a real content-correlated split (digit vs. word tokens on one seed), though the specific feature wasn't identical on the other seed. Layer 6 specifically (the depth-progression layer) wasn't re-checked at the final checkpoint — still open.
- Should `nu_sep` (currently unchanged at 0.05) be measured the same way `mu_load` was (§6.2)? `sep_loss` showed real instability of its own (swings from -3.5 to -10.6 within ~120 steps in the collapsed run) that was not directly investigated — it may be a downstream symptom of the same collapse dynamic, or a separate miscalibration.
- The self-tuning (per-point adaptive) bandwidth design discussed but not implemented (the "Option 2" alternative to §2.2's global live-refresh) remains a candidate if staleness-driven instability recurs even with the current fix.
- `NystromDiffusionMap`'s own k-means/eigensolve randomness still has a hardcoded `random_state=42`, independent of `cfg.seed` (§9) — the seed-repeat check in §6.5 varied model init and data-adjacent RNG state, but not this. Worth wiring through too if the seed-sensitivity finding in §6.5 needs isolating further (is it driven by init, by the diffusion-map's own landmark selection, or both).